

CS Study

Database - 트랜잭션 격리수준

목차

1

트랜잭션 격리 수준이란

2

데이터 관련 문제 현상

3

트랜잭션 격리 수준 종류

4

성능 관련

목차

1

트랜잭션 격리 수준이란

2

데이터 관련 문제 현상

3

트랜잭션 격리 수준 종류

4

성능 관련

트랜잭션 격리 수준이란?

DB 특성

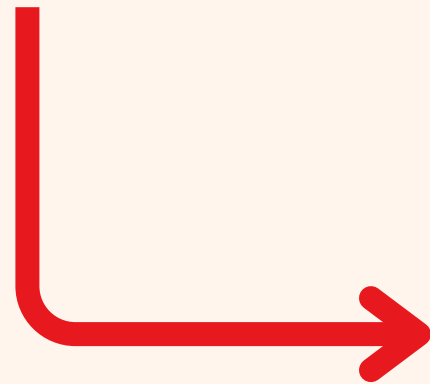
- 여러 사용자가 동시에 데이터 Read or Write
- 같은 데이터 접근 시 데이터 일관성 깨질 수 있음



트랜잭션 격리 수준이란?

데이터 일관성 vs 성능

- 데이터 일관성 유지 → **트랜잭션 순서대로** → 성능 하락 →
- **트랜잭션 동시 처리** → 성능 상승 → 잘못된 값 처리 확률 상승



트랜잭션 격리 수준이란?

여러 종류 격리 수준 제공

데이터 일관성 vs 성능 사이 선택



트랜잭션 격리 수준이란?

트랜잭션 격리 수준

- 각 트랜잭션이 다른 트랜잭션의
작업 결과 언제 읽을 수 있는지
정하는 것

격리 수준	Dirty Read	Unrepeatable Read	Phantom Data Read
READ UNCOMMITTED	O	O	O
READ COMMITTED	X	O	O
REPEATABLE READ	X	X	O
SERIALIZABLE	X	X	X

목차

1

트랜잭션 격리 수준이란

2

데이터 관련 문제 현상

3

트랜잭션 격리 수준 종류

4

성능 관련

데이터 관련 문제 현상

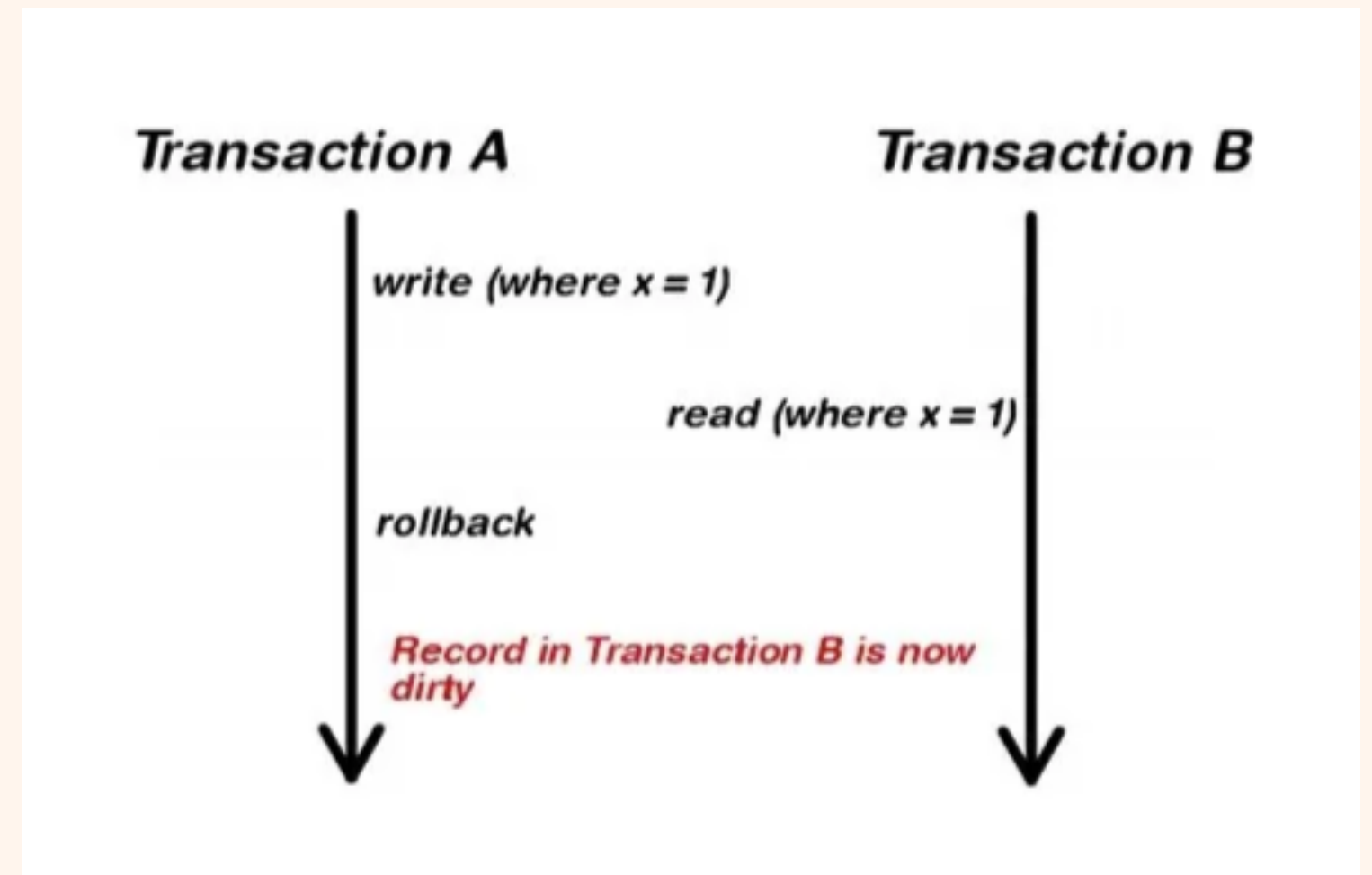
Dirty Read

- A 트랜잭션 **아직 COMMIT X 데이터**
B 트랜잭션 **읽게 허용해서** 읽는 현상
- READ_UNCOMMITTED에서 발생함
- READ_COMMITTED 부터는 발생 X

데이터 관련 문제 현상

Dirty Read

- A 트랜잭션에서 **ROLLBACK** 하면 문제 발생
- B 트랜잭션에서 **잘못된 데이터 읽고 처리**
- **데이터 일관성 문제**가 됨



데이터 관련 문제 현상

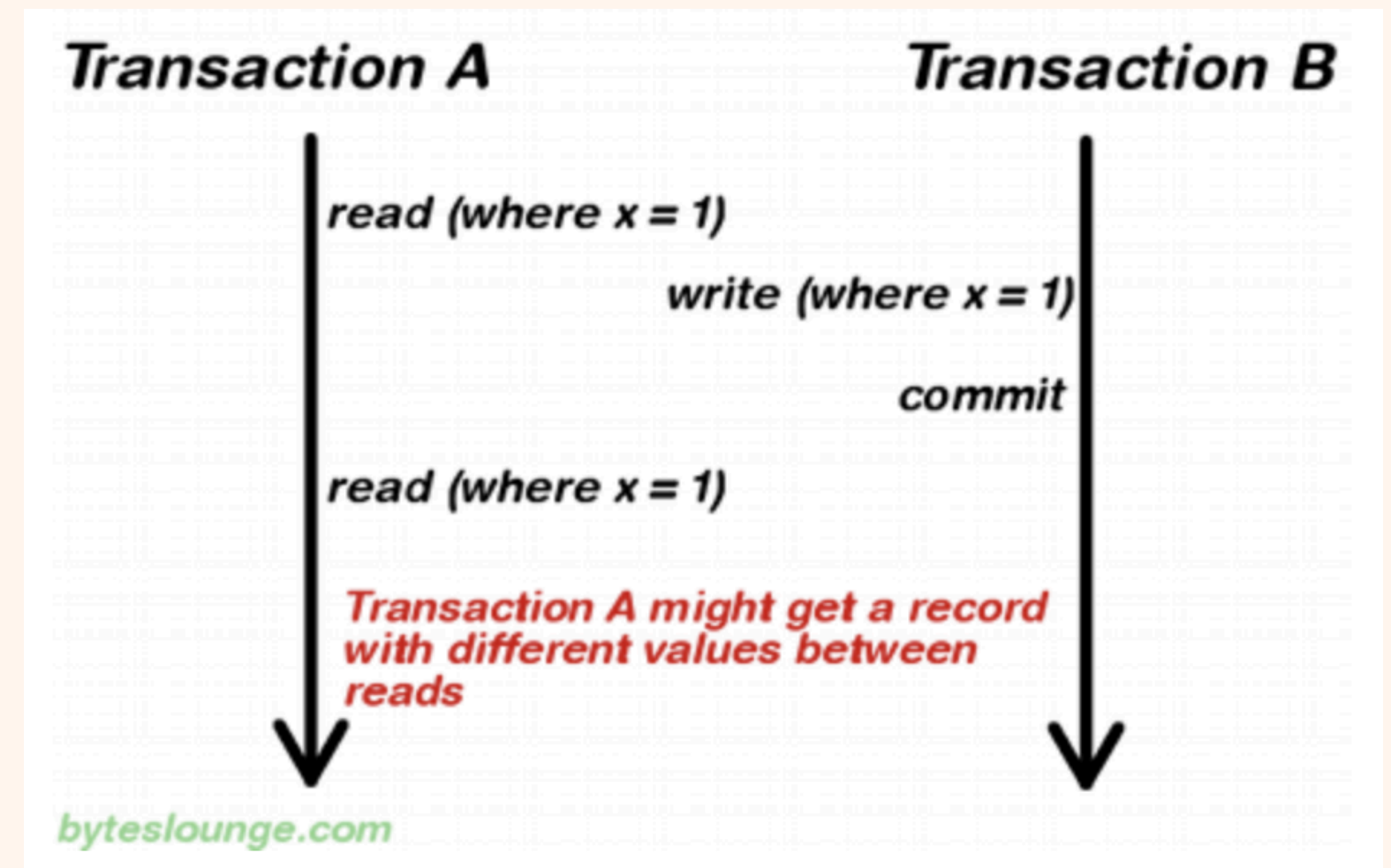
Unrepeatable Read

- A 트랜잭션 다음 **SELECT** 쿼리 실행 시
이전과 결과 다른 일관성 깨지는 문제
- READ_COMMITTED 까지 발생함
- REPEATABLE_READ 부터는 발생 X

데이터 관련 문제 현상

Unrepeatable Read

- A 트랜잭션 **반복적으로 SELECT 쿼리 실행**
- SELECT 쿼리 **중간에** B 트랜잭션이 데이터 **수정, 삭제하고 COMMIT**
- 처음 읽은 값과 **나중에 읽은 값 일치 X**



데이터 관련 문제 현상

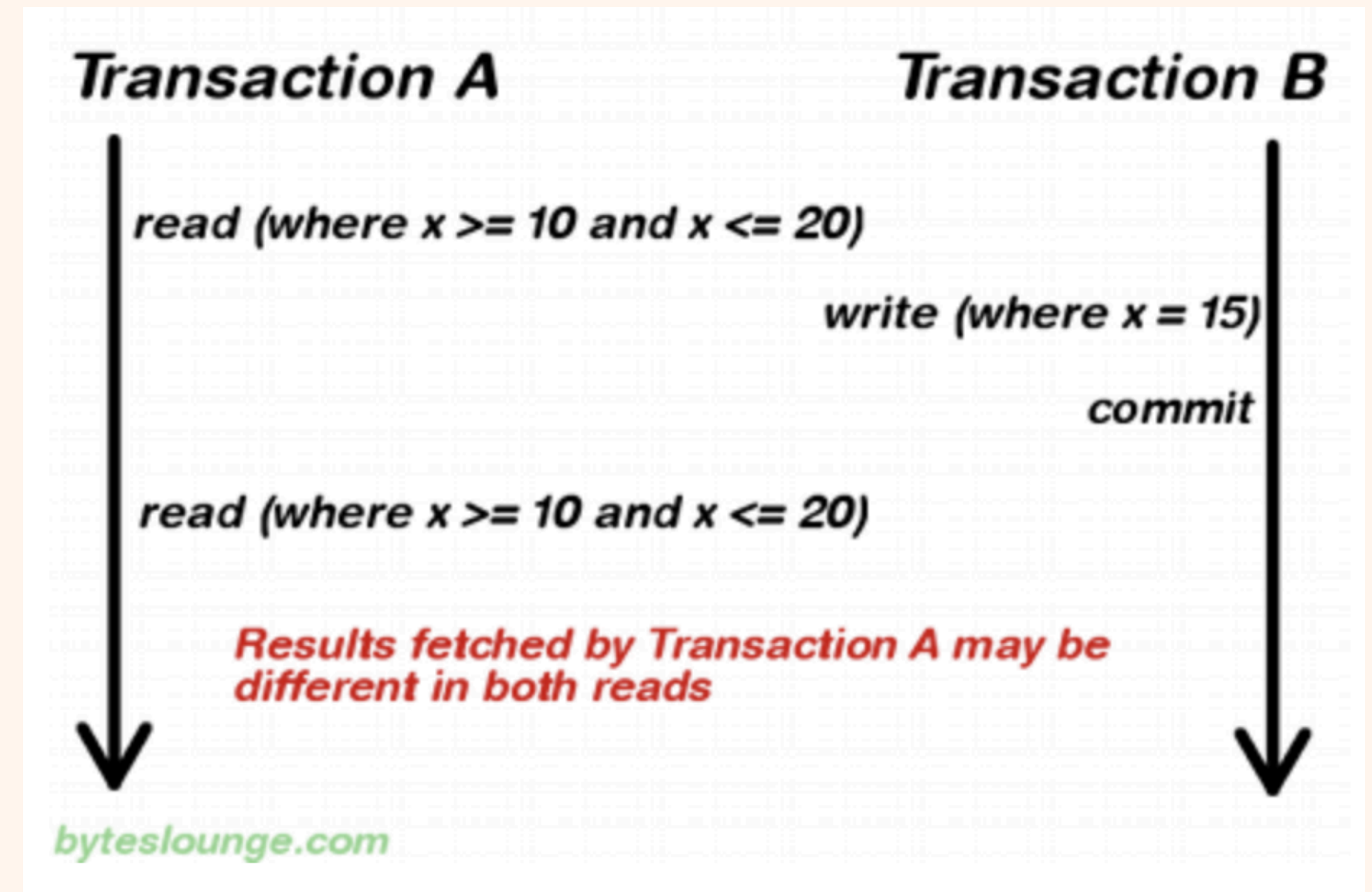
Phantom Data Read

- A 트랜잭션 다음 조회 시 값들 사라지고 나타나는 일관성 깨지는 문제
- REPEATABLE READ 까지 발생함
- SERIALIZABLE 부터는 발생 X

데이터 관련 문제 현상

Phantom Data Read

- A 트랜잭션 **특정 범위의 데이터**
여러 번 조회
- 조회 중간에 B 트랜잭션 **특정 범위**
데이터 INSERT
- 트랜잭션 **중간에 새로운 데이터**
INSERT 허용해 나타나는 문제



목차

1

트랜잭션 격리 수준이란

2

데이터 관련 문제 현상

3

트랜잭션 격리 수준 종류

4

성능 관련

트랜잭션 격리 수준 종류

트랜잭션 격리 수준 종류

- READ_UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

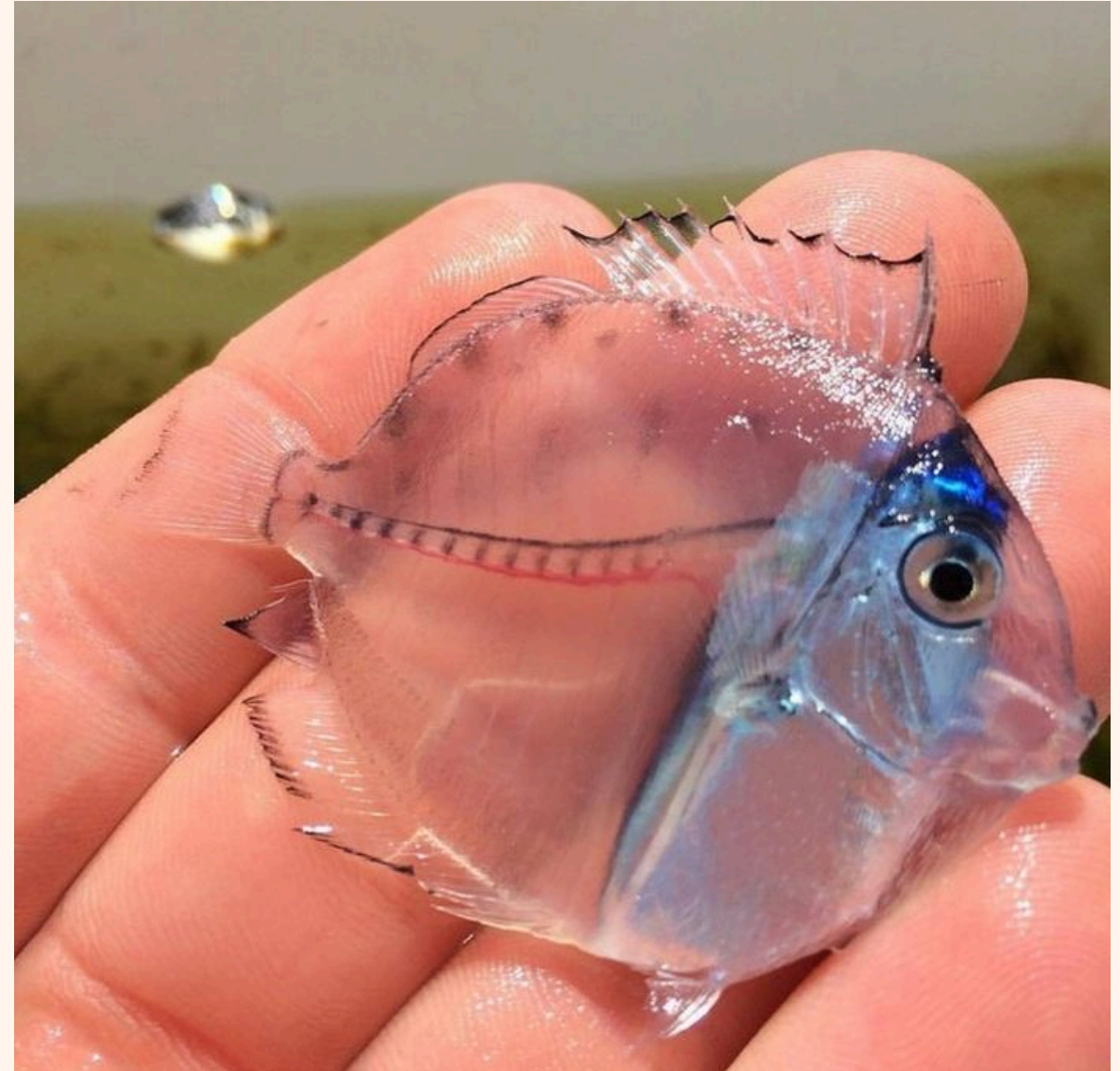
격리 수준 높아짐



트랜잭션 격리 수준 종류

READ_UNCOMMITTED

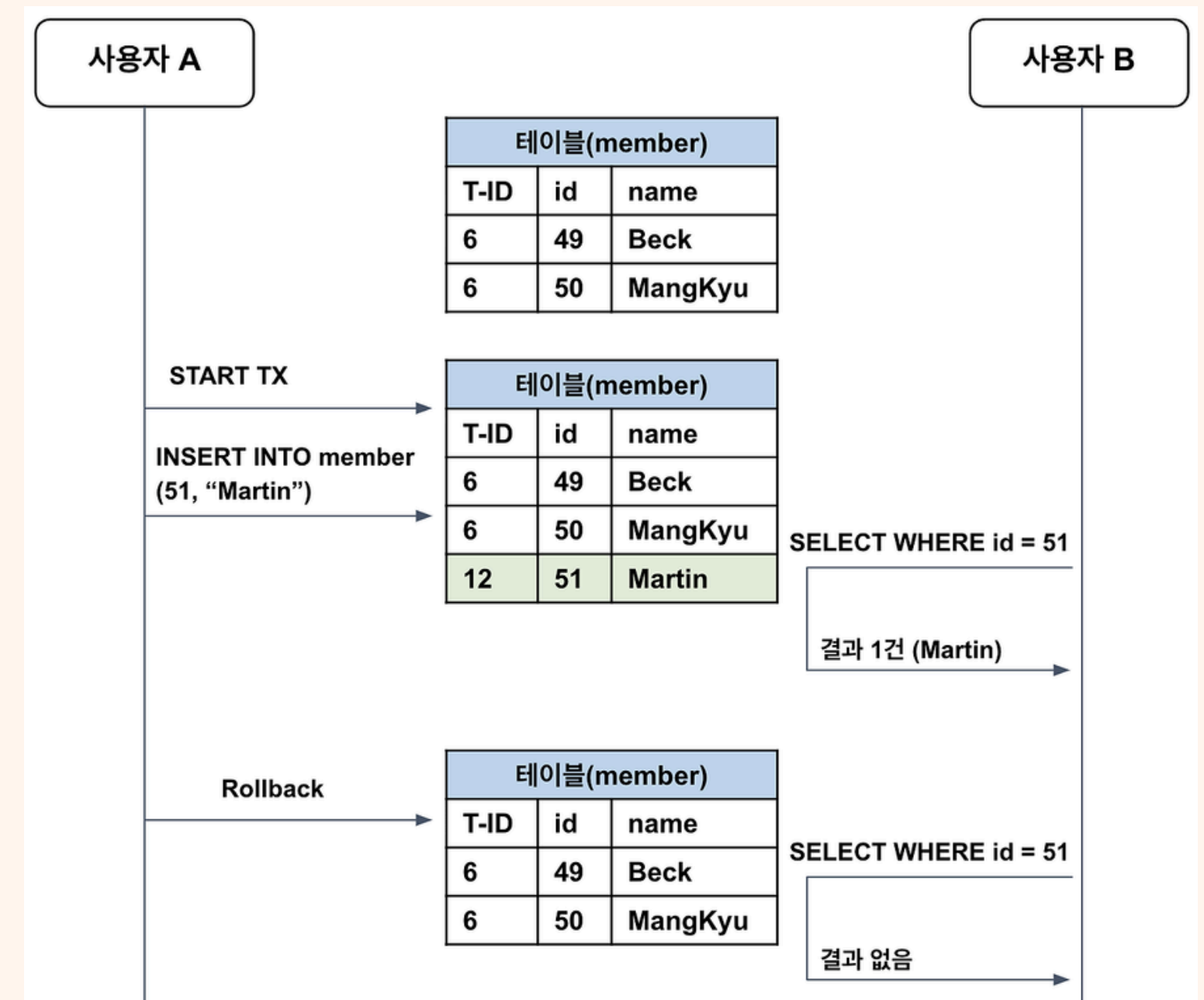
- 다른 트랜잭션 **COMMIT** 되지 않은
변경 데이터조차 접근할 수 있는 격리 수준
- **Dirty Read 발생**, 상당한 버그 초래 가능



트랜잭션 격리 수준 종류

READ_UNCOMMITTED

- A 트랜잭션 INSERT 후 COMMIT X
- B 트랜잭션 INSERT된 값 바로 보임
- A 트랜잭션 ROLLBACK 시 없는 값 됨
- → Dirty Read 문제



트랜잭션 격리 수준 종류

READ_UNCOMMITTED

- **성능상으로는** 가장 좋고 동시성 높음
- **데이터 일관성면**에서 문제 많음
- 대부분의 DB **기본으로 사용 X, 권장 X**



트랜잭션 격리 수준 종류

READ_COMMITTED

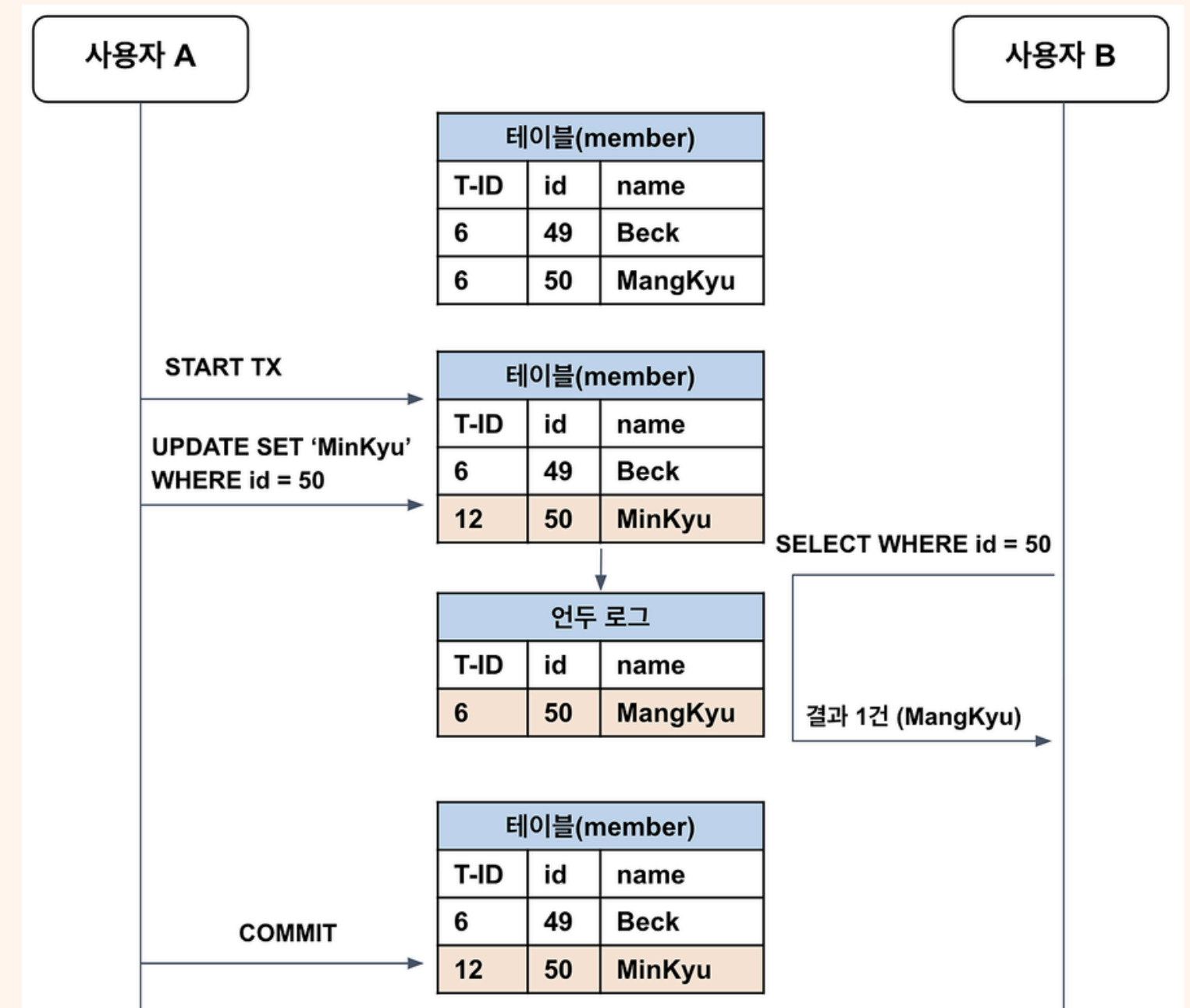
- 다른 트랜잭션 **COMMIT**한 변경
데이터만 읽을 수 있는 격리 수준
- **Dirty Read 방지**



트랜잭션 격리 수준 종류

READ_COMMITTED

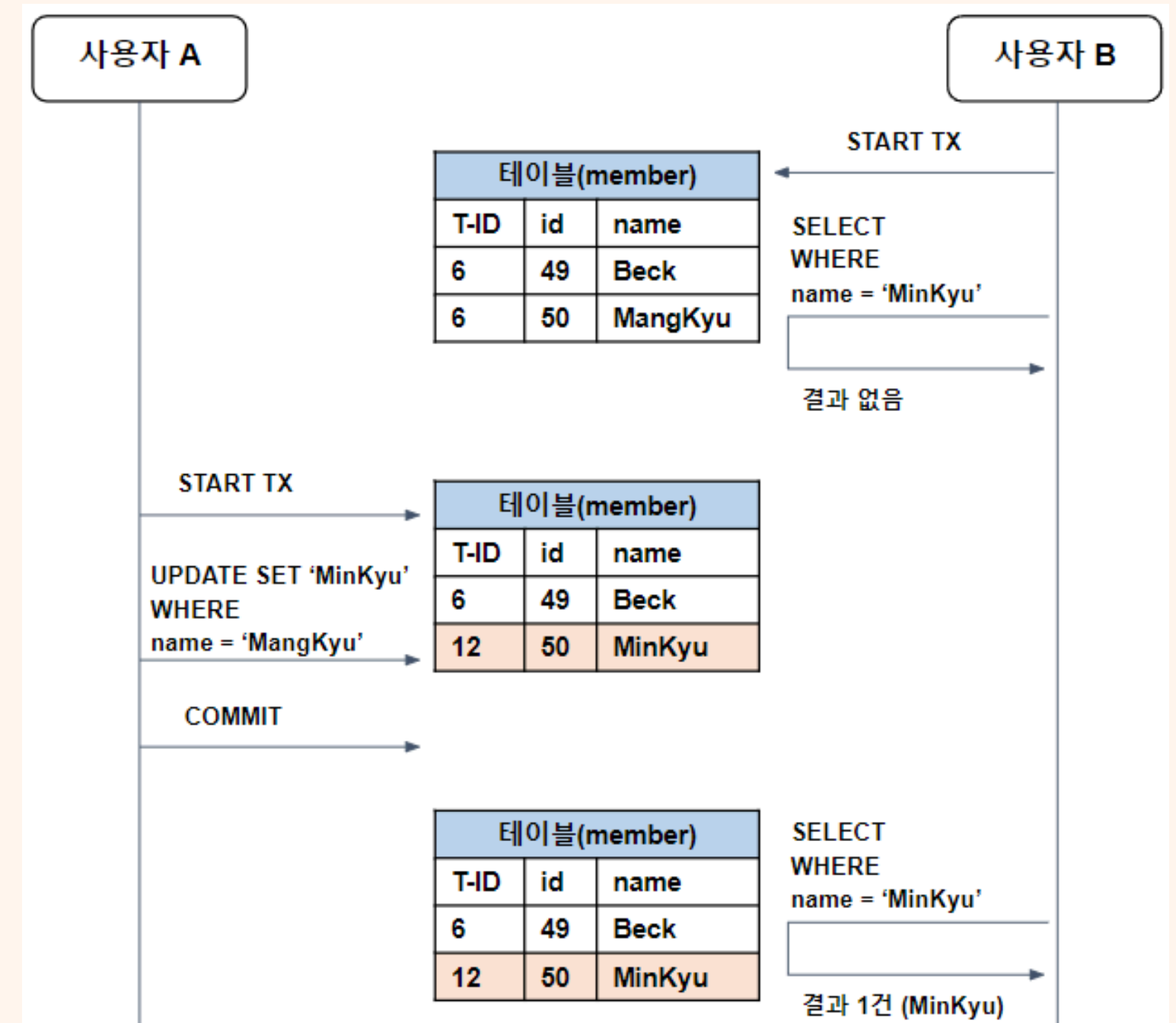
- A 트랜잭션 값 수정 시 테이블 갱신
- **undo log에 변경 전 데이터 백업**
- B 트랜잭션 undo log서 변경 전 데이터 확인
- **SELECT 수행되는 동안** 해당 데이터에 **Shared Lock**



트랜잭션 격리 수준 종류

READ_COMMITTED

- 트랜잭션 중 **다른 트랜잭션 데이터 수정 막지 X**
- 트랜잭션 중 **다시 데이터 읽으면 값 변경됨**
- → **Non Repeatable Read** 문제
- 다른 트랜잭션 **중간에 값 삽입, 삭제해 결과 변화**
- → **Phantom Read** 문제



트랜잭션 격리 수준 종류

READ_COMMITTED

- 가장 많이 사용되는 격리 수준
- 많은 DB의 **Default**
- 성능과 데이터 일관성 간 **균형**
- **일반 CRUD, 조회 많은 서비스**에서 적합한 선택



트랜잭션 격리 수준 종류

REPEATABLE_READ

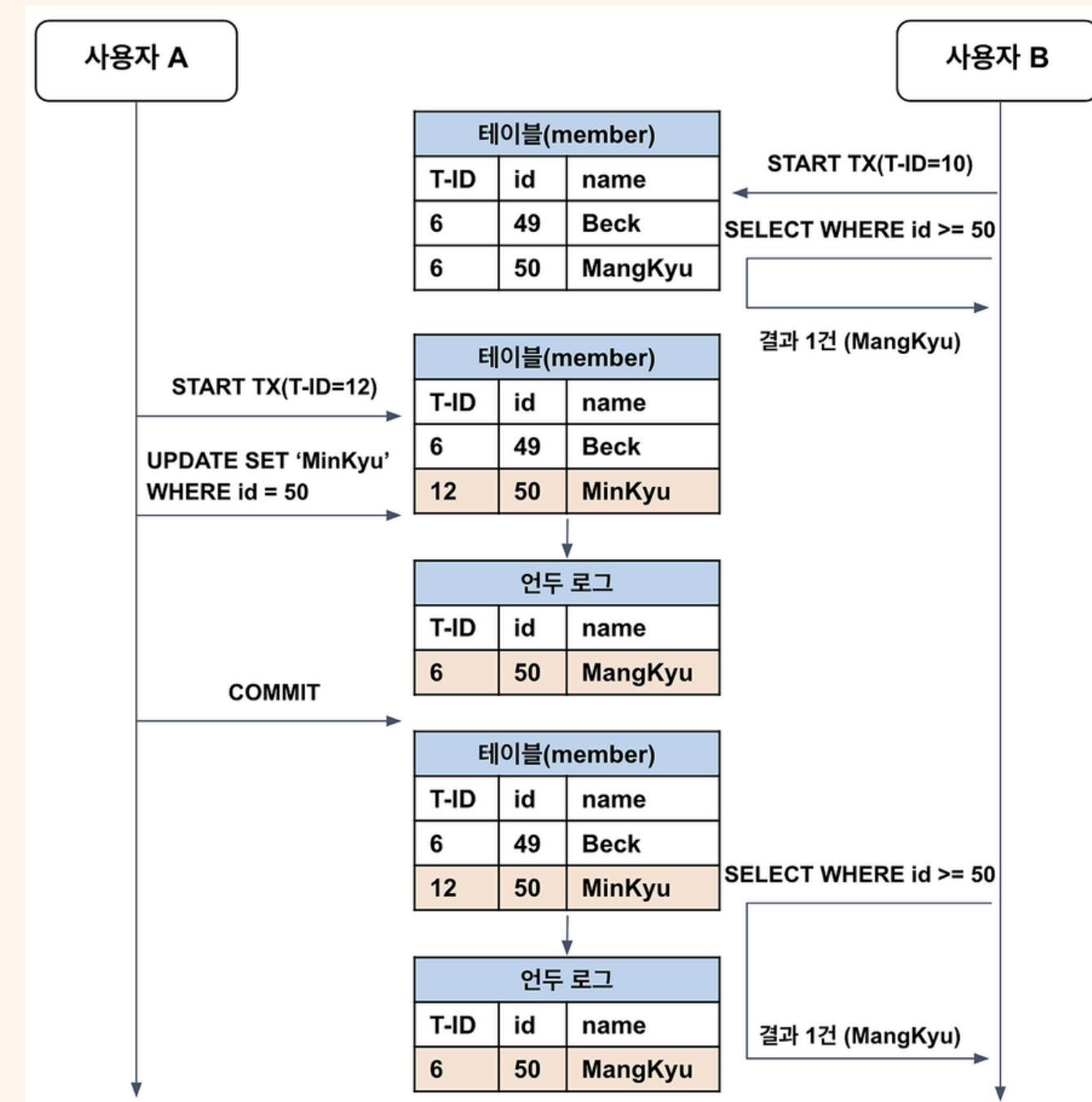
- **트랜잭션 완료까지** SELECT문
사용 **모든 데이터에 Shared Lock**
- 트랜잭션 범위 **조회 내용 동일 보장**



트랜잭션 격리 수준 종류

REPEATABLE_READ

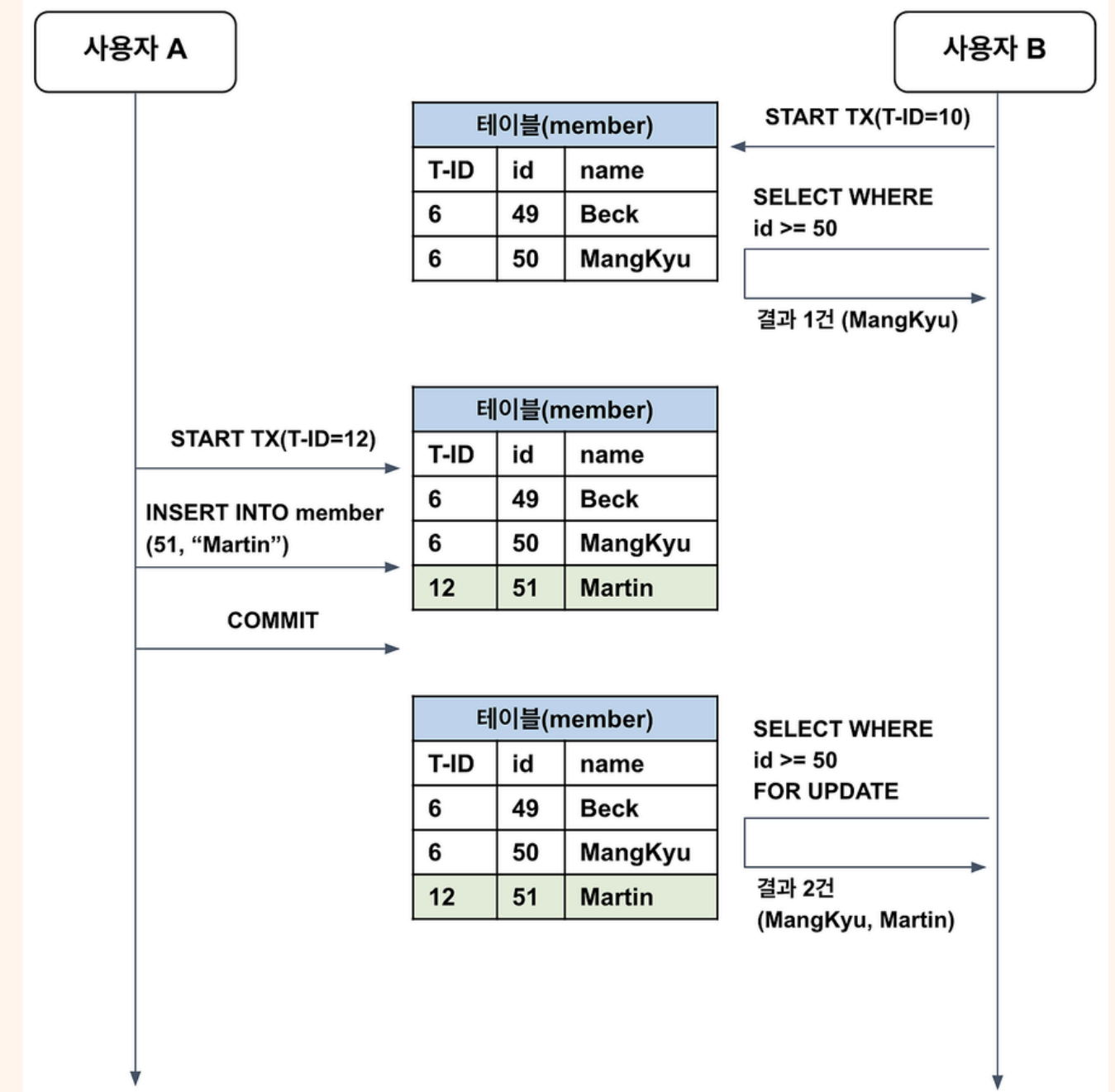
- 변경 전의 레코드 undo log에 백업
- 각 **트랜잭션에 고유 번호** 존재
- **백업 데이터에** 트랜잭션 번호 **함께 저장**
- **먼저 실행된** 트랜잭션의 데이터만 조회
- **트랜잭션 실행 순간** shared lock
- → Non Repeatable Read 문제 해결



트랜잭션 격리 수준 종류

REPEATABLE_READ

- 새로운 데이터 추가되는 경우 문제
- Lock 사용 SELECT 시 undo log에서 읽기 X
- 실제 테이블에서 읽어와 중간에 추가되면 문제
- → Phantom Read 문제
- 이 경우 제외 MVCC, Gap Lock 등으로 해결



트랜잭션 격리 수준 종류

SERIALIZABLE

- 순차적으로 실행되는 것과 같은 결과 보장하는 격리 수준
- 모든 이상 현상 완벽히 차단



트랜잭션 격리 수준 종류

SERIALIZABLE

- **트랜잭션 접근 데이터**에 동시에 접근, 수정, 추가, 삭제 불가능
- **모든 쿼리에 Lock 걸어** 다른 트랜잭션 수정, 삽입, 삭제 불가능
- Phantom Read 방지해야 돼서
트랜잭션 전체에 대해 잠금 지속



트랜잭션 격리 수준 종류

SERIALIZABLE

- 트랜잭션 직렬화 처리해 **동시 처리 성능 가장 떨어짐**
- 다른 트랜잭션 접근 시 **대기, DeadLock 발생 위험 높아짐**
- 금융, 회계 같이 **데이터 일관성 100% 보장**해야 할 때 사용
- 성능 Trade Off 극심해 **매우 제한적 사용**



목차

1

트랜잭션 격리 수준이란

2

데이터 관련 문제 현상

3

트랜잭션 격리 수준 종류

4

성능 관련

성능 관련

트랜잭션 격리 수준과 성능

- 트랜잭션 격리 수준은 **성능에 매우 중요한 고려사항**
- 데이터 일관성과 동시 처리 성능 **사이에서 고민해 선택**

트랜잭션 격리 수준	데이터 일관성	동시 처리 성능
높음	강함	낮음
낮음	약함	높음

성능 관련

격리 수준별 성능 특성

격리 수준	성능 특성	동시성 문제/트랜잭션 충돌	주의점
READ UNCOMMITTED	가장 빠름 (락 거의 없음)	매우 높음	Dirty Read, 데이터 신뢰성 문제
READ COMMITTED	빠름, 대부분 서비스 기본	높음	Non-repeatable Read, Phantom Read 발생 가능
REPEATABLE READ	성능 적당, 일관성 보장 (읽기는 스냅샷, 쓰기는 락)	중~높음	Gap Lock 등으로 일부 트랜잭션 간 충돌 가능
SERIALIZABLE	가장 느림, 직렬 처리에 가까움	낮음	응답 지연, DeadLock 발생 가능

성능 관련

성능에 영향 주는 요소

Lock 사용

- 격리 수준 높을수록 많은 데이터에 Lock
- 동시 처리 트랜잭션 줄어들어 성능 저하

MVCC 사용

- 읽기 작업에서 Lock을 적게 감
- Repeatable에 사용해도 동시성 높이기 가능 늘어나 처리량 감소

Deadlock

- 높은 격리 수준에서 위험 증가
- Lock 많고 오래 유지되면 위험 증가

트랜잭션 지속시간

- 트랜잭션 길어지면 Lock 점유 시간

성능 관련

Tip

READ_COMMITTED

- 읽기 성능 중요한 경우 사용
- Lock 대기 없이 빠른 대규모 조회 가능

대규모 쓰기 작업 or 동시성 중요 시

- 트랜잭션 짧게 유지
- 필요 없는 격리 수준은 사용 X

REPEATABLE_READ 이상

- 데이터 정합성 중요한 경우
- 필요 이상으로 트랜잭션 유지 안하게 주의

Serializable 수준 최소화

- 극히 일부분에서 필요할 때만 잠깐 사용

Quiz

1번 문제

READ_COMMITTED 격리 수준에선

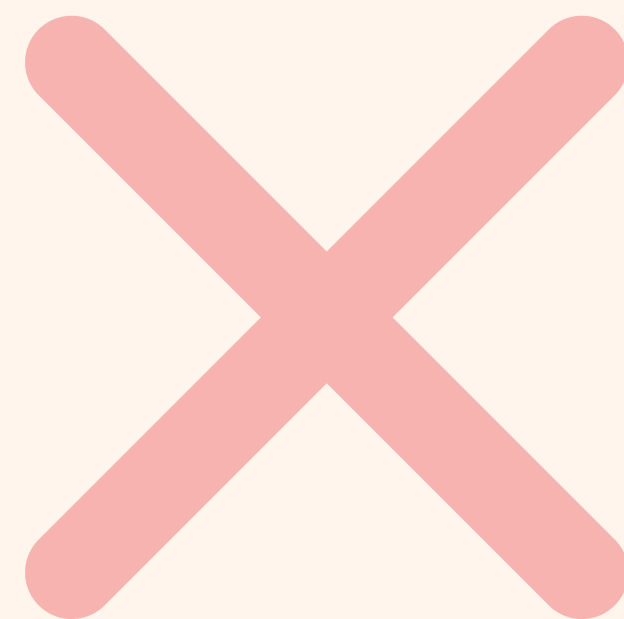
Dirty Read가 일어나지 않는다.



1번 문제

Dirty Read는 **Commit 되지 않은 데이터**에도
접근 가능해 발생하는 문제로

READ_UNCOMMITTED 수준에서만 발생



2번 문제

격리 수준이 높아질수록
_____는 강해지고 _____는 낮아진다

2번 문제

격리 수준이 높아질수록 Lock의 개수나
적용 범위가 넓어져 데이터 일관성이 높아짐

그에 따라 격리 수준이 높아질수록
트랜잭션 동시 처리 성능이 낮아짐

3번 문제

**READ_COMMITTED에서 트랜잭션 중간에
다른 트랜잭션의 데이터 쓰기를 막는다.**



3번 문제

READ_COMMITTED에서 중간에 다른

트랜잭션의 **데이터 쓰기**를 막지 **않아**

Unrepeatable Read 발생

